



UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

SEDE PORTALES

INGENIERÍA EN SISTEMAS

ANÁLISIS DE SISTEMAS II

ING. ANTONIO ALVARADO

OCTAVO SEMESTRE DE 2026

SECCIÓN “A”

ENTREGABLE 2:

Requerimientos Ágiles, BDD, UI y Desarrollo Inicial de la API

Integrantes	Carné
Victoriano Gómez Pérez	9989-23-18022
Dulce María Morales Ramírez	9989-23-18927
Jose Pablo Valdez de León	9989-23-85
Nidia Alejandra Alvizures Alvizures	9989-23-14166

GUATEMALA, 02 DE OCTUBRE DE 2026

1. INTRODUCCION

El presente documento corresponde al Entregable 2 del proyecto Sistema de Gestión de Compras para Farmacias Galeno, cuyo propósito es definir y documentar los requerimientos ágiles, las especificaciones funcionales, el diseño de la experiencia de usuario y el desarrollo inicial de la API que conformará la base tecnológica del sistema.

Este entregable integra prácticas de metodologías ágiles mediante la elaboración de un Product Backlog priorizado, la definición de historias de usuario y criterios de aceptación bajo el enfoque Behavior Driven Development (BDD). Asimismo, se desarrolla el diseño de interfaces de usuario mediante prototipos interactivos que permiten visualizar la navegación y la interacción con las funcionalidades principales del sistema.

Adicionalmente, se establece la estructura inicial del backend, incluyendo la configuración de la base de datos y la implementación de los primeros endpoints operativos para la gestión de información. De esta manera, se busca garantizar una base sólida para el desarrollo incremental del sistema, asegurando la trazabilidad entre los requerimientos del negocio, el diseño de la solución y la implementación técnica.

2. REQUERIMIENTOS ÁGILES Y PRODUCT BACKLOG

2.1 Descripción de Épicas y Mínimo Producto Viable (MVP)

2.1.1. Descripción general

El Product Backlog del Sistema de Gestión de Compras para Farmacias Galeno se organiza en épicas e historias de usuario para estructurar las funcionalidades principales del sistema. Se definieron cuatro épicas: gestión de compras y abastecimiento, gestión de proveedores y catálogo, control de inventario y lotes farmacéuticos, y gestión de cuentas por pagar. Además, se incluye la autenticación de usuarios como una función transversal que permite el acceso al sistema según los permisos asignados.

2.1.2. Descripción de Épicas

Épica 01 – Gestión de Abastecimiento y Compras a Proveedores

Descripción:

Esta épica comprende las funcionalidades relacionadas con el proceso de abastecimiento y generación de órdenes de compra a proveedores. Su objetivo es permitir que el encargado de compras pueda crear y validar órdenes de compra, seleccionando proveedores, medicamentos y cantidades requeridas.

Dentro de esta épica se incluyen las siguientes historias de usuario:

- **HU-04 – Crear orden de compra**
- **HU-05 – Validar datos de orden de compra**
- **HU-06 – Generar número de orden de compra**

Estas funcionalidades permiten establecer el flujo necesario para registrar una orden de compra de manera estructurada y mantener un identificador que facilite su seguimiento.

Épica 02 – Gestión de Catálogo y Proveedores

Descripción:

Esta épica comprende las funcionalidades relacionadas con el registro y administración de los proveedores y la relación de los medicamentos con los proveedores disponibles. Su objetivo es

mantener organizada y actualizada la información necesaria para facilitar el proceso de abastecimiento y selección de proveedores.

Dentro de esta épica se incluyen las siguientes historias de usuario:

- **HU-02 – Registrar proveedor**
- **HU-03 – Vincular medicamentos con proveedor**

Estas funcionalidades permiten mantener la información de los proveedores y establecer qué medicamentos puede suministrar cada uno, facilitando posteriormente la creación de órdenes de compra.

Épica 03 – Control de Inventario y Lotes Farmacéuticos

Descripción:

Esta épica comprende las funcionalidades relacionadas con la recepción de medicamentos y el control de las existencias, lotes y fechas de vencimiento.

Su objetivo es permitir que el encargado de inventario registre correctamente los medicamentos recibidos y mantenga actualizada la información de las existencias, incluyendo los datos necesarios para el control de los lotes farmacéuticos.

Dentro de esta épica se incluyen:

- **HU-07 – Registrar recepción de medicamentos**
- **HU-08 – Registrar lote y fecha de vencimiento**
- **HU-09 – Actualizar existencias del inventario**

Estas funcionalidades permiten dar continuidad al proceso iniciado mediante una orden de compra y registrar los productos efectivamente recibidos.

Épica 04 – Gestión de Cuentas por Pagar a Proveedores

Descripción:

Esta épica comprende las funcionalidades relacionadas con el registro y validación de las facturas emitidas por los proveedores y su relación con las órdenes de compra correspondientes.

Su objetivo es mantener la trazabilidad entre las facturas y las órdenes de compra, evitando que se registren obligaciones relacionadas con órdenes inexistentes o que no correspondan al proveedor de la factura.

relacionadas con órdenes inexistentes o que no correspondan al proveedor de la factura.

Dentro de esta épica se incluyen:

- **HU-10 – Registrar factura de proveedor**
- **HU-11 – Vincular factura con orden de compra**
- **HU-12 – Validar orden relacionada con factura**

Estas funcionalidades permiten completar el flujo administrativo de la compra mediante el registro de la documentación correspondiente.

Funcionalidad transversal – Autenticación de usuarios

El sistema contempla además una funcionalidad transversal de consulta de información, destinada a permitir que los usuarios autorizados puedan consultar los datos registrados relacionados con compras, proveedores e inventario.

Esta funcionalidad se encuentra representada mediante:

- **HU-01 – Inicio de sesión de usuario**

La autenticación constituye un punto de entrada para las funcionalidades del sistema y se relaciona con los diferentes roles definidos para la gestión de las operaciones.

2.1.3. Mínimo Producto Viable (MVP)

El Mínimo Producto Viable (MVP) del Sistema de Gestión de Compras para Farmacias Galeno está conformado por las funcionalidades necesarias para ejecutar el flujo principal de gestión de compras y abastecimiento.

El MVP contempla desde el acceso de los usuarios al sistema, pasando por la gestión de proveedores y la creación de órdenes de compra, hasta la recepción de medicamentos, actualización del inventario y registro y validación de las facturas de proveedores.

El flujo principal del MVP se representa de la siguiente manera:

Inicio de sesión → Gestión de proveedores → Orden de compra → Recepción de medicamentos → Control de lotes y vencimientos → Actualización de inventario → Registro de factura → Validación de cuenta por pagar

Las historias de usuario consideradas como parte del MVP corresponden principalmente a las historias priorizadas como Must, debido a que representan funcionalidades necesarias para el funcionamiento del flujo principal del sistema.

Por lo tanto, el MVP está compuesto por:

ID	Historia de Usuario	Prioridad
HU-01	Inicio de sesión de usuario	Must
HU-02	Registrar proveedor	Must
HU-03	Vincular medicamentos con proveedor	Must
HU-04	Crear orden de compra	Must
HU-05	Validar datos de orden de compra	Must
HU-06	Generar número de orden de compra	Must
HU-07	Registrar recepción de medicamentos	Must
HU-08	Registrar lote y fecha de vencimiento	Must
HU-09	Actualizar existencias del inventario	Must
HU-10	Registrar factura de proveedor	Must
HU-11	Vincular factura con orden de compra	Must
HU-12	Validar orden relacionada con factura	Must

La HU-13 – Consultar información del sistema se mantiene como una funcionalidad de prioridad Should, por lo que no forma parte del conjunto mínimo indispensable del MVP.

2.2 Criterios de Priorización y Estimación (Story Points)

Para la construcción del Product Backlog del Sistema de Gestión de Compras para Farmacias Galeno se establecieron criterios de priorización y estimación que permiten organizar las historias de usuario de acuerdo con su importancia para el funcionamiento del sistema y el esfuerzo relativo requerido para su implementación.

2.2.1. Criterio de Priorización MoSCoW

El método MoSCoW permite clasificar las historias de usuario según el nivel de importancia que tienen dentro del producto y, especialmente, dentro del Mínimo Producto Viable (MVP).

Las categorías utilizadas son:

Prioridad	Descripción
Must	Funcionalidad indispensable para el funcionamiento del sistema y para cumplir el alcance mínimo del MVP.
Should	Funcionalidad importante que aporta valor al sistema, pero que puede implementarse después de las funcionalidades indispensables.
Could	Funcionalidad deseable que puede incorporarse si existe tiempo y capacidad disponible después de completar las funcionalidades prioritarias.
Won't	Funcionalidad que no será desarrollada dentro del alcance actual del proyecto, aunque pueda considerarse para futuras versiones.

Aplicación de MoSCoW al proyecto

Para este proyecto, las historias HU-01 a HU-12 fueron clasificadas como Must, debido a que forman parte del flujo principal requerido para gestionar proveedores, compras, recepción de medicamentos, inventario y cuentas por pagar.

La HU-13 fue clasificada como Should, debido a que la consulta general de información aporta valor al sistema, pero no constituye una operación indispensable para completar el flujo básico del MVP.

2.2.2. Criterio de Estimación mediante Story Points

La estimación de las historias de usuario se realiza mediante Story Points, utilizando una escala basada en la sucesión de Fibonacci:

1, 2, 3, 5, 8, 13

Los Story Points representan una estimación relativa del esfuerzo necesario para implementar una historia de usuario. No representan directamente horas de trabajo.

Para determinar los puntos de cada historia se consideran principalmente los siguientes factores:

- **Complejidad:** dificultad técnica y funcional de la implementación.
- **Esfuerzo:** cantidad de trabajo necesario para completar la funcionalidad.
- **Dependencias:** relación de la historia con otros componentes o funcionalidades del sistema.
- **Validaciones:** cantidad y complejidad de las reglas que deben cumplirse.
- **Incertidumbre:** nivel de conocimiento o dificultad para determinar anticipadamente la solución.

De esta manera, una historia con 3 Story Points representa una complejidad y esfuerzo relativo menor que una historia de 5 Story Points, mientras que una historia de 5 puntos requiere mayor esfuerzo relativo que una de 3 puntos.

2.2.3. Criterios utilizados para la asignación de Story Points

Para mantener una estimación consistente se utilizaron los siguientes criterios:

Story Points	Criterio general
1	Funcionalidad muy sencilla, con poca lógica y sin dependencias relevantes.
2	Funcionalidad sencilla con algunas validaciones o interacción básica.
3	Funcionalidad de complejidad baja-media, con algunas validaciones y reglas de negocio.
5	Funcionalidad de complejidad media, con varias reglas, validaciones o dependencias entre componentes.
8	Funcionalidad compleja que involucra múltiples procesos o dependencias significativas.
13	Funcionalidad de alta complejidad o con un nivel considerable de incertidumbre; podría requerir dividirse en historias más pequeñas.

Para el Product Backlog actual se utilizaron principalmente 3 y 5 Story Points, debido a que las historias definidas corresponden a funcionalidades acotadas del sistema y no presentan, en su alcance actual, una complejidad suficiente para requerir estimaciones de 8 o 13 puntos.

2.2.4. Estimación de las Historias de Usuario

La siguiente tabla presenta la estimación asignada a cada historia de usuario:

2.3 Tabla Consolidada de Historias de Usuario:

La siguiente tabla consolida las historias de usuario definidas para el sistema, indicando su identificación, descripción, épica asociada, prioridad según MoSCoW y estimación mediante Story Points.

El seguimiento y gestión del Product Backlog se realiza mediante GitHub Projects:

[\[Abrir Tablero Ágil\]](#)

2.4. Reglas de negocio

Código	Regla de negocio	HU relacionada
RN-01	El usuario debe autenticarse para acceder al sistema.	HU-01
RN-02	Para registrar un proveedor deben completarse los datos obligatorios.	HU-02
RN-03	Un medicamento solo puede vincularse con un proveedor registrado.	HU-03
RN-04	Una orden de compra debe estar asociada a un proveedor registrado.	HU-04
RN-05	Los datos de la orden deben validarse antes de registrarla.	HU-05
RN-06	Cada orden de compra debe tener un número único.	HU-06
RN-07	Una recepción debe estar asociada a una orden de compra existente.	HU-07
RN-08	Todo lote debe registrar número de lote y fecha de vencimiento válida.	HU-08
RN-09	Las existencias deben actualizarse de acuerdo con las cantidades recibidas.	HU-09

RN-10	Una factura debe corresponder a un proveedor registrado.	HU-10
RN-11	Una factura debe poder vincularse únicamente con una orden de compra existente.	HU-11
RN-12	Los datos de la factura deben coincidir con la orden relacionada para aprobar su validación.	HU-12
RN-13	La consulta de información requiere permisos de acceso correspondientes.	HU-13

3. ESPECIFICACIONES BDD (CRITERIOS DE ACEPTACIÓN)

3.1. Cobertura de Escenarios Gherkin

Las especificaciones BDD del Sistema de Gestión de Compras para Farmacias Galeno se desarrollan a partir de las historias de usuario definidas en el Product Backlog. Cada historia de usuario contará con criterios de aceptación expresados mediante escenarios Gherkin, permitiendo validar tanto el comportamiento esperado del sistema como las situaciones de error o excepción.

Para garantizar la cobertura de los requerimientos, cada historia de usuario tendrá como mínimo:

- Un escenario principal o camino feliz, que representa la ejecución exitosa de la funcionalidad.
- Un escenario de excepción o alternativo, que permite validar el comportamiento del sistema ante datos inválidos, información inexistente, operaciones no permitidas u **otras** condiciones que impidan completar correctamente la operación.

Los escenarios utilizarán una estructura Gherkin en inglés basada en los elementos:

- Feature: identifica la funcionalidad que se desea validar.
- Scenario: representa una situación específica que debe comprobarse.
- Given: establece las condiciones iniciales del escenario.
- When: describe la acción realizada por el usuario o por el sistema.
- Then: establece el resultado esperado.

- And: permite agregar condiciones o acciones adicionales al escenario.

La cobertura se realizará sobre las 13 historias de usuario ya definidas en el Product Backlog, manteniendo los mismos identificadores y funcionalidades para asegurar la trazabilidad entre ambos componentes.

La distribución de las historias de usuario por archivo. feature será la siguiente:

Archivo .feature	Historias de Usuario	Funcionalidad
autenticacion.feature	HU-01	Autenticación de usuarios
proveedores.feature	HU-02, HU-03	Gestión de catálogo y proveedores
compras.feature	HU-04, HU-05, HU-06	Gestión de abastecimiento y compras
inventario.feature	HU-07, HU-08, HU-09	Control de inventario y lotes farmacéuticos
cuentas_por_pagar.feature	HU-10, HU-11, HU-12	Gestión de cuentas por pagar

De esta manera, los escenarios Gherkin permitirán establecer criterios de aceptación verificables para las historias de usuario y mantener la trazabilidad entre los requerimientos, el Product Backlog y las pruebas de comportamiento.

Cobertura prevista

Tipo de escenario	Cobertura mínima
Historias de Usuario	13
Escenarios principales	13

Escenarios de excepción/alternativos	14
Total mínimo de escenarios	27

3.2. Matriz de Escenario por historia

La matriz de escenarios permite establecer la relación entre cada Historia de Usuario definida en el Product Backlog y las especificaciones BDD correspondientes. Para cada historia se define el archivo .feature donde se documentarán sus escenarios, así como la cantidad de escenarios felices y de error considerados para validar el comportamiento esperado del sistema.

Los escenarios felices representan el flujo principal de cada funcionalidad, es decir, aquellas situaciones en las que el usuario proporciona información válida y la operación se completa correctamente. Por otro lado, los escenarios de error permiten comprobar el comportamiento del sistema ante situaciones como datos inválidos, información incompleta, registros inexistentes, credenciales incorrectas o condiciones que impidan completar la operación.

La distribución de los escenarios se organiza de acuerdo con las funcionalidades establecidas en las cuatro épicas principales y las historias de usuario definidas anteriormente. De esta manera, se mantiene la trazabilidad entre el Product Backlog y los criterios de aceptación desarrollados mediante Gherkin.

Historia de Usuario	Archivo .feature	Escenarios Felices	Escenarios de Error
HU-01	features/autenticacion.feature	1	2
HU-02	features/proveedores.feature	1	1
HU-03	features/proveedores.feature	1	1
HU-04	features/compras.feature	1	1

HU-05	features/compras.feature	1	1
HU-06	features/compras.feature	1	1
HU-07	features/inventario.feature	1	1
HU-08	features/inventario.feature	1	1
HU-09	features/inventario.feature	1	1
HU-10	features/cuentas_por_pagar.feature e	1	1
HU-11	features/cuentas_por_pagar.feature e	1	1
HU-12	features/cuentas_por_pagar.feature e	1	1
HU-13	features/consultas.feature	1	1

Vínculo a Archivos .feature: [features/](#)

4. DISEÑO DE EXPERIENCIA DE USUARIO (UX/UI)

El diseño de experiencia de usuario (UX/UI) del Sistema de Gestión de Compras para Farmacias Galeno tiene como finalidad proporcionar una interfaz clara, organizada, responsive y fácil de utilizar para los usuarios encargados de gestionar proveedores, medicamentos, órdenes de compra, recepción de productos, inventario y cuentas por pagar.

Para el desarrollo del prototipo interactivo se utilizó Tailwind CSS, permitiendo construir una interfaz adaptable a diferentes tamaños de pantalla y representar visualmente los principales módulos y flujos definidos en los requerimientos ágiles y en el MVP del proyecto.

El diseño toma como referencia las historias de usuario HU-01 a HU-13 y el flujo principal del MVP: inicio de sesión, gestión de proveedores, creación de órdenes de compra, recepción de medicamentos, registro de lotes y fechas de vencimiento, actualización del inventario y gestión de facturas y cuentas por pagar.

4.1 Mapa de Navegación del Sistema

El mapa de navegación representa la estructura principal del Sistema de Gestión de Compras para Farmacias Galeno. El usuario inicia sesión y posteriormente accede al Dashboard, desde donde puede ingresar a los diferentes módulos.

Nivel	Pantalla / módulo	HU relacionada
1	Inicio de sesión	HU-01
2	Dashboard	HU-13
3	Proveedores	HU-02, HU-03
3	Medicamentos	HU-03
3	Órdenes de compra	HU-04, HU-05, HU-06
3	Recepción	HU-07, HU-08
3	Inventario	HU-09
3	Cuentas por pagar	HU-10, HU-11, HU-12
3	Consultas	HU-13

El flujo principal de navegación corresponde al proceso definido para el MVP: Proveedor → Orden de compra → Recepción → Lote y vencimiento → Actualización de inventario → Factura → Validación de cuentas por pagar.

El prototipo representa esta navegación mediante un menú principal y pantallas independientes para cada módulo. Esto permite que el usuario siga una secuencia lógica y tenga acceso directo a las funciones principales.

4.2 Mapeo de Pantallas vs. Endpoints OpenAPI

El diseño UX/UI se relaciona con la futura implementación de la API mediante un mapeo entre las pantallas del prototipo y las operaciones que deberán ser atendidas por los servicios backend.

La siguiente relación permite identificar qué funcionalidad de la interfaz deberá comunicarse posteriormente con la API:

Pantalla / módulo	Historia de usuario	Operación esperada de la API
Inicio de sesión	HU-01	Autenticación y validación de credenciales

Dashboard / Consultas	HU-13	Consulta de información del sistema
Proveedores	HU-02	Registro y consulta de proveedores
Medicamentos	HU-03	Consulta y relación de medicamentos con proveedores
Órdenes de compra	HU-04, HU-05, HU-06	Creación, validación y generación del número de orden
Recepción	HU-07	Registro de recepción de medicamentos
Lote y vencimiento	HU-08	Registro de lote y fecha de vencimiento
Inventario	HU-09	Consulta y actualización de existencias
Cuentas por pagar	HU-10	Registro de factura
Cuentas por pagar	HU-11	Relación de factura con orden de compra
Cuentas por pagar	HU-12	Validación de la orden relacionada
Consultas	HU-13	Consulta de información almacenada

En esta etapa, el prototipo representa la relación funcional entre las pantallas y los servicios de la API.

Los nombres y rutas definitivas de los endpoints serán establecidos de acuerdo con el contrato

OpenAPI desarrollado para el backend del proyecto.

Por esta razón, el prototipo no establece rutas definitivas que no hayan sido definidas en el contrato de la API. La sección de trazabilidad del prototipo permite identificar la relación entre cada módulo de la interfaz, la historia de usuario correspondiente y el servicio que deberá consumir cuando la API sea integrada.

Esta estructura facilita posteriormente la conexión entre frontend y backend, debido a que cada funcionalidad visual ya se encuentra asociada con una necesidad funcional específica del sistema.

4.3 Reporte de Evaluación de Usabilidad y Accesibilidad (WCAG 2.1)

La evaluación de usabilidad y accesibilidad se realizó tomando como referencia principios de las Web Content Accessibility Guidelines (WCAG) 2.1, con el objetivo de que la interfaz pueda ser utilizada de manera clara y consistente por los usuarios del sistema.

Para el prototipo se consideraron los siguientes aspectos:

4.3.1 Perceptibilidad

La información se presenta mediante una estructura visual organizada, utilizando títulos, etiquetas, botones y secciones claramente diferenciadas.

Se utilizaron elementos visuales que permiten identificar las diferentes funciones del sistema, como:

- Menú lateral para la navegación.
- Títulos y subtítulos para identificar cada módulo.
- Tablas para representar información estructurada.
- Formularios con etiquetas asociadas a los campos.
- Mensajes visuales para indicar estados o resultados de operaciones.
- Indicadores de estado para facilitar la interpretación de la información.

La interfaz también utiliza un diseño responsive mediante Tailwind CSS, permitiendo que los elementos se adapten a diferentes tamaños de pantalla.

4.3.2 Operabilidad

La navegación del prototipo se encuentra organizada mediante un menú principal que permite acceder directamente a los módulos del sistema.

Los botones y controles se presentan con etiquetas descriptivas, evitando depender únicamente de elementos gráficos para identificar una acción.

Las principales acciones del sistema se encuentran representadas mediante botones como:

- Nuevo proveedor.

- Nueva orden.
- Registrar recepción.
- Actualizar inventario.
- Registrar factura.
- Consultar información.

Esto permite que el usuario pueda identificar fácilmente las acciones disponibles en cada pantalla.

4.3.3 Comprensibilidad

La interfaz mantiene una estructura consistente entre los diferentes módulos. Los formularios utilizan etiquetas descriptivas y los elementos se organizan de acuerdo con el flujo de trabajo correspondiente.

La información se presenta de forma progresiva, evitando colocar todos los datos y acciones en una única pantalla.

Además, las operaciones se organizan de acuerdo con el proceso establecido en el MVP, permitiendo que el usuario siga una secuencia lógica desde la gestión del proveedor hasta la validación de las cuentas por pagar.

4.3.4 Robustez

El prototipo fue desarrollado utilizando HTML y Tailwind CSS, manteniendo una estructura basada en componentes estándar de la web.

La interfaz está preparada para posteriormente integrarse con los servicios de la API mediante solicitudes HTTP. De esta manera, las pantallas pueden evolucionar desde una demostración visual hacia una aplicación funcional conectada con el backend.

4.3.5 Diseño Responsive

Uno de los objetivos del prototipo es permitir que la interfaz pueda visualizarse correctamente tanto en computadoras como en dispositivos con pantallas de menor tamaño.

Para ello, Tailwind CSS permite utilizar clases responsive que modifican la distribución de los elementos según el tamaño de la pantalla.

En dispositivos de escritorio se utiliza una estructura con menú lateral y área principal de trabajo, mientras que en pantallas pequeñas los elementos se reorganizan para facilitar su visualización y navegación.

Esto permite que el sistema mantenga una experiencia de uso consistente en diferentes dispositivos.

4.3.6 Evaluación de criterios de accesibilidad

Criterio	Aplicación en el prototipo	Resultado
Perceptibilidad	Títulos, etiquetas, tablas, indicadores y estructura visual organizada.	Cumple
Operabilidad	Menú de navegación y controles con textos descriptivos.	Cumple
Comprensibilidad	Estructura consistente y formularios con etiquetas claras.	Cumple
Robustez	HTML estándar y estructura preparada para integración con API.	Cumple
Diseño responsive	Adaptación a escritorio y dispositivos móviles mediante Tailwind CSS.	Cumple
Contraste visual	Diferenciación entre fondos, textos, estados y acciones.	Cumple
Navegación por teclado	Uso de controles HTML estándar; requiere validación manual completa.	Parcial

La evaluación anterior corresponde a una revisión inicial del prototipo y no sustituye una auditoría formal de accesibilidad. Para una validación completa sería necesario realizar pruebas adicionales con herramientas automáticas y pruebas manuales de accesibilidad, incluyendo navegación mediante teclado, lectores de pantalla y diferentes tamaños de pantalla.

4.4 Prototipo Interactivo

Como evidencia del componente UX/UI se desarrolló un prototipo interactivo utilizando Tailwind CSS. El prototipo representa visualmente las principales funcionalidades del Sistema de Gestión de Compras para Farmacias Galeno.

Pantalla	Funcionalidad principal
Inicio de sesión	Acceso al sistema mediante credenciales de demostración.
Dashboard	Resumen de proveedores, órdenes, inventario y facturas.
Proveedores	Consulta y registro visual de proveedores.
Medicamentos	Consulta y relación de medicamentos con proveedores.
Órdenes de compra	Visualización y creación de órdenes.
Recepción	Registro de recepción, lote y vencimiento.
Inventario	Consulta y actualización de existencias.
Cuentas por pagar	Registro y validación de facturas.
Trazabilidad UX/API	Relación entre pantallas, HUs y servicios de API.
Accesibilidad	Visualización de criterios considerados para WCAG 2.1.

Las credenciales utilizadas exclusivamente para la demostración del prototipo son:

Usuario: admin

Contraseña: 123456

Estas credenciales corresponden únicamente al prototipo de demostración y no representan credenciales reales del sistema productivo.

4.5 Relación del Diseño UX/UI con las Historias de Usuario

HU	Funcionalidad representada en UX/UI
HU-01	Inicio de sesión y acceso al sistema.
HU-02	Gestión de proveedores.
HU-03	Relación de medicamentos con proveedores.
HU-04	Creación de órdenes de compra.
HU-05	Validación de datos de la orden.
HU-06	Generación del número de orden.

HU-07	Registro de recepción de medicamentos.
HU-08	Registro de lote y fecha de vencimiento.
HU-09	Actualización del inventario.
HU-10	Registro de facturas.
HU-11	Relación de factura con orden de compra.
HU-12	Validación de la orden relacionada.
HU-13	Consulta de información del sistema.

4.6 Conclusión del Diseño UX/UI

El componente UX/UI establece una representación visual del Sistema de Gestión de Compras para Farmacias Galeno antes de completar la integración con el backend. El prototipo desarrollado con Tailwind CSS organiza las funcionalidades de acuerdo con el flujo definido en el MVP y proporciona una estructura de navegación clara entre los módulos de proveedores, medicamentos, compras, recepción, inventario y cuentas por pagar.

Además, la relación entre historias de usuario, pantallas y futuras operaciones de API facilita la trazabilidad del proyecto y permite que el diseño sirva como referencia para las etapas de desarrollo e integración.

Finalmente, se consideraron criterios de usabilidad, diseño responsive y principios de accesibilidad basados en WCAG 2.1, con el propósito de proporcionar una interfaz clara, consistente y adaptable a las necesidades del sistema.

Evidencia del prototipo

El archivo HTML del prototipo interactivo se entrega como evidencia complementaria del apartado 4. El prototipo permite iniciar sesión con las credenciales de demostración y navegar por los módulos desarrollados.

5. ARQUITECTURA DEL CASCARÓN BACKEND (BOILERPLATE)

Explicación de las capas del prototipo

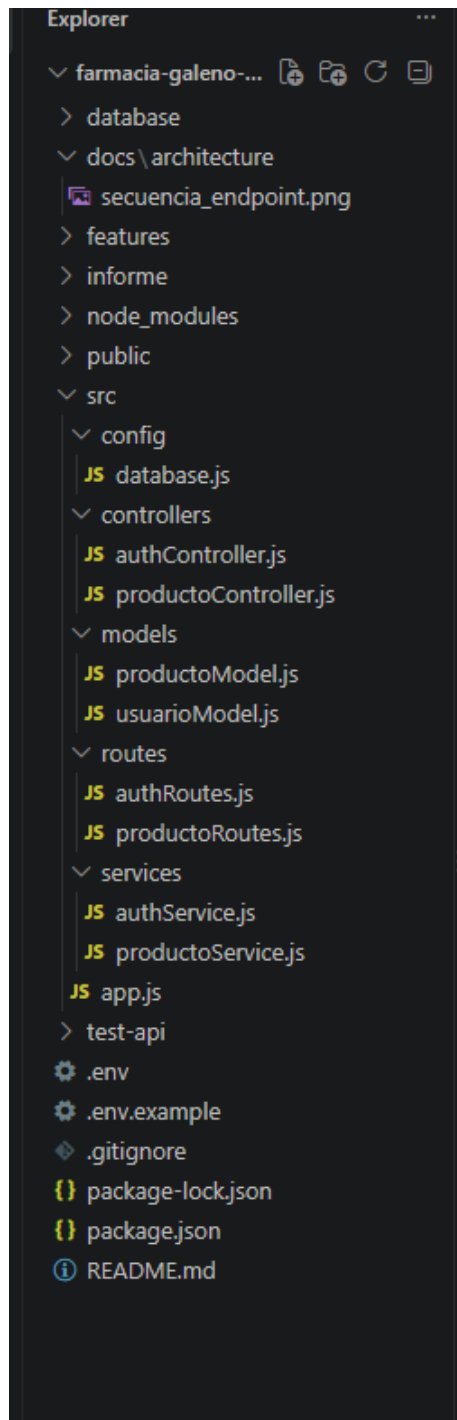
El proyecto Farmacia Galeno API utiliza una arquitectura backend organizada por capas, con el objetivo de separar las diferentes responsabilidades del sistema y facilitar su mantenimiento, comprensión y futuras modificaciones.

La estructura se encuentra organizada dentro de la carpeta **src**, donde cada componente cumple una función específica:

- **config**: contiene la configuración necesaria para el funcionamiento de la aplicación, principalmente la conexión con la base de datos.
- **routes**: define las rutas o endpoints disponibles en la API y determina qué controlador debe atender cada solicitud.
- **controllers**: recibe las solicitudes HTTP provenientes del cliente, procesa los parámetros necesarios y coordina la respuesta que será enviada.
- **services**: contiene la lógica de negocio de la aplicación, separando las operaciones que debe realizar el sistema de la lógica propia de las rutas y controladores.
- **models**: se encarga de las operaciones relacionadas con los datos y la interacción con la base de datos MySQL.

Esta separación permite que cada capa tenga una responsabilidad específica. De esta manera, los cambios realizados en una parte del sistema pueden realizarse sin afectar directamente a las demás capas. Además, esta organización facilita las pruebas, el mantenimiento y la ampliación de nuevos endpoints.

La estructura general del backend es:

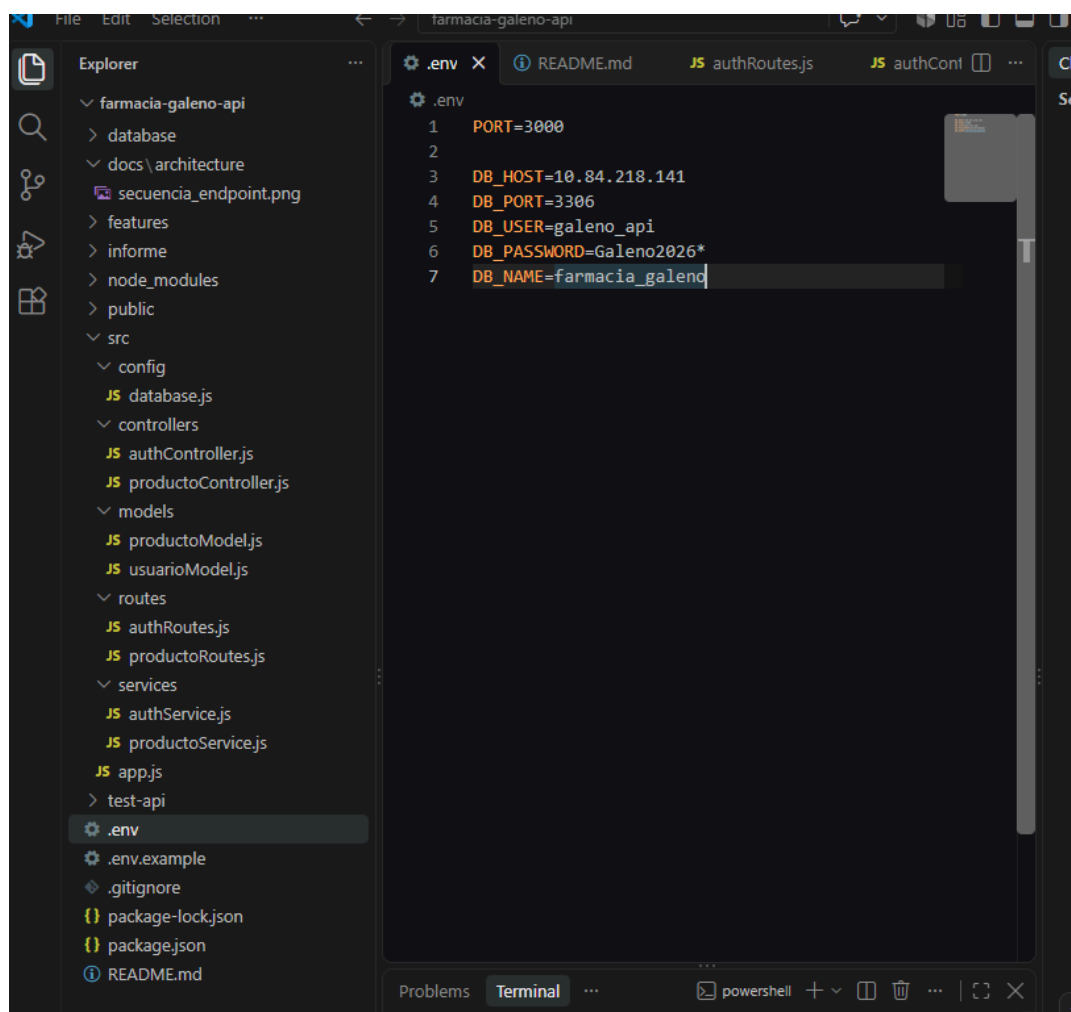


Configuración de conexión a base de datos

El backend de Farmacia Galeno API utiliza una base de datos MySQL para almacenar y consultar la información utilizada por los diferentes endpoints de la aplicación.

La configuración de la conexión se mantiene separada de la lógica de los controladores y servicios, permitiendo que las credenciales y parámetros de conexión sean administrados mediante variables de entorno.

Para este propósito se utiliza un archivo `.env`, donde se establecen los datos necesarios para realizar la conexión, tales como el servidor, puerto, usuario, contraseña y nombre de la base de datos.



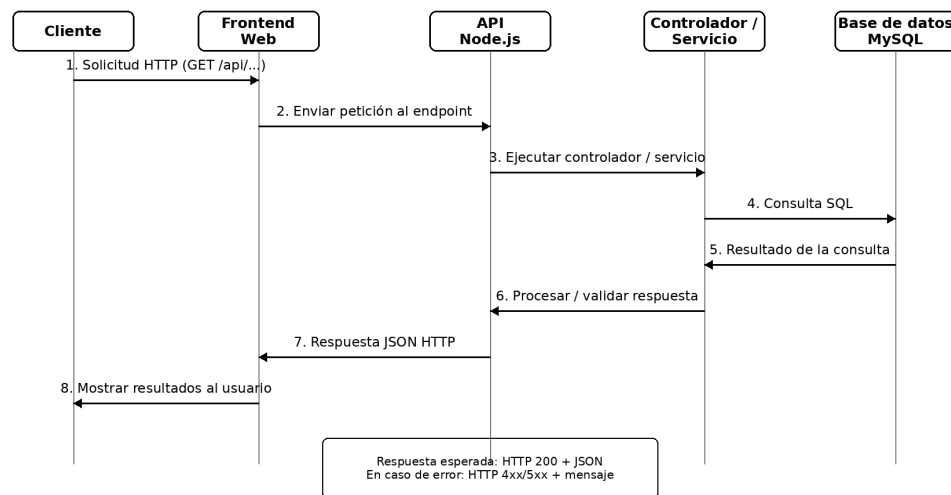
The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows the project structure for 'farmacia-galeno-api'. The code editor displays the contents of the `.env` file, which contains the following configuration:

```
.env
1  PORT=3000
2
3  DB_HOST=10.84.218.141
4  DB_PORT=3306
5  DB_USER=galeno_api
6  DB_PASSWORD=Galeno2026*
7  DB_NAME=farmacia_galeno
```


La aplicación obtiene estos valores al iniciar el servidor y utiliza la configuración establecida en la capa correspondiente para establecer la comunicación con MySQL.

Esta implementación permite evitar colocar directamente las credenciales de la base de datos dentro del código fuente, facilitando además la configuración del sistema en diferentes ambientes.

Diagrama de secuencia - API FARMACIA GALENO



6. IMPLEMENTACIÓN Y PRUEBA DE ENDPOINTS INICIALES

GET /api/v1/productos

Cuando el cliente realiza una solicitud a este endpoint, la petición es recibida por la ruta correspondiente. Posteriormente, el controlador coordina la operación con la capa de servicios, que ejecuta la lógica necesaria para realizar la consulta a través del modelo y obtener la información almacenada en MySQL.

Finalmente, los datos obtenidos son procesados y enviados al cliente mediante una respuesta HTTP en formato JSON.

POST /api/v1/productos

Este endpoint recibe mediante el cuerpo de la solicitud HTTP los datos correspondientes al producto que se desea registrar.

La solicitud es recibida inicialmente por la ruta correspondiente y posteriormente enviada al controlador. El controlador delega la operación a la capa de servicios, donde se ejecuta la lógica necesaria antes de realizar la operación de inserción mediante el modelo.

Finalmente, el modelo ejecuta la operación correspondiente sobre la base de datos MySQL y el resultado es retornado hacia el cliente mediante una respuesta JSON.

Postman interface showing a GET request to `http://localhost:3000/api/v1/auth/usuarios`. The response is a 200 OK status with a JSON body:

```
1 {
2   "success": true,
3   "count": 1,
4   "data": [
5     {
6       "id": 1,
7       "nombre": "Pablo",
8       "email": "pablo@firemaria.com",
9       "rol": "ADMIN",
10      "estado": 1
11    }
12  ]
13 }
```

The right sidebar shows a chat window with the following text:

Debugging /api/v1/auth/usuarios...

Let me start by examining the "Usuarios" request and the collection structure to understand what's available.

Thought for 24 seconds

Retrieved details for request Usuarios from tab

Found collection farmacia-galeano-api

Thought for 38 seconds

Let me also check the other requests to understand the API's route patterns better.

Retrieved HTTP request: Login correcto

Retrieved HTTP request: Productos

+ Generating...

Postman interface showing a POST request to `http://localhost:3000/api/v1/auth/login`. The request body is:

```
1 {
2   "email": "pablo@firemaria.com",
3   "password": "v1Pab10n000123"
4 }
```

The response is a 401 Unauthorized status with a JSON body:

```
1 {
2   "success": false,
3   "message": "Error de autenticación",
4   "cause": "Contraseña incorrecta"
5 }
```

The right sidebar shows a chat window with the following text:

New Chat

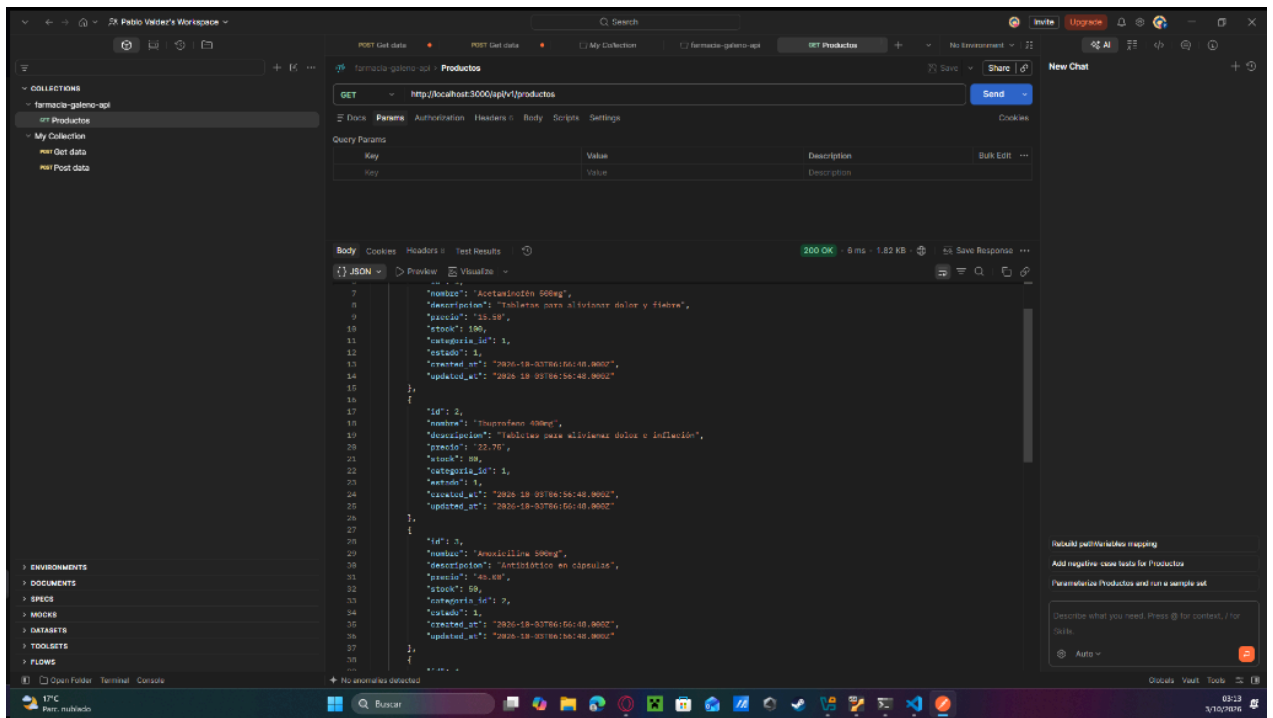
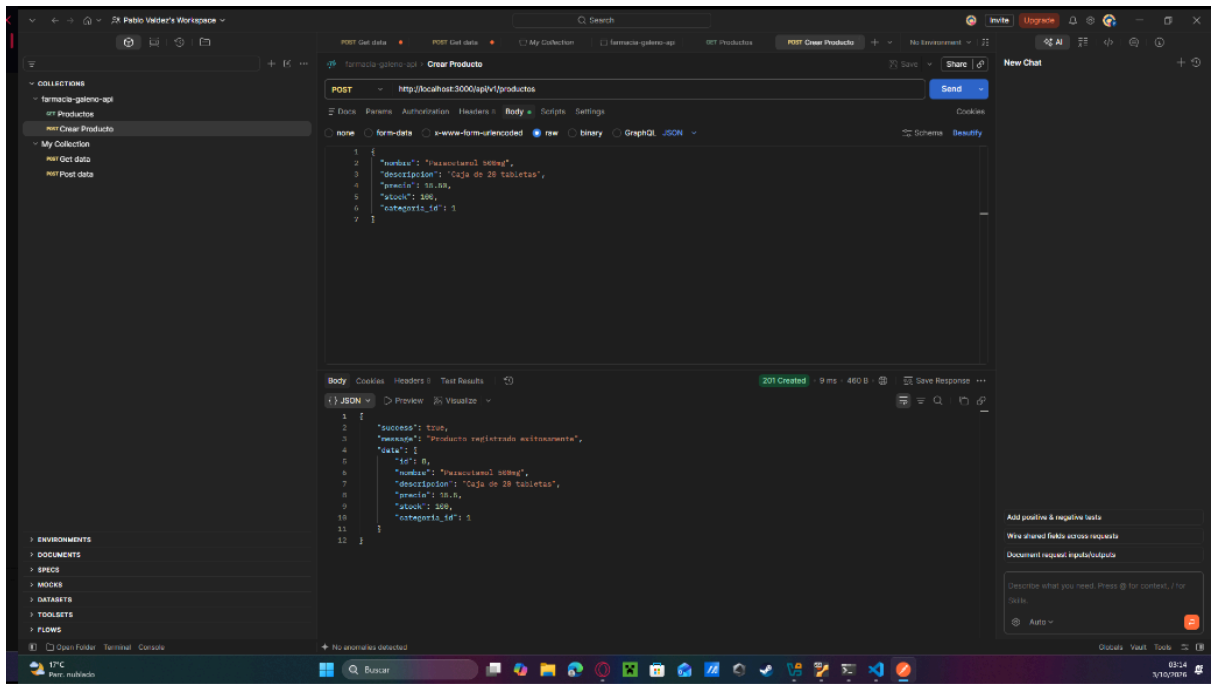
Add test assertions for login

Create negative login tests

Parameterize credentials with variables

Describe what you need. Press @ for context, / for help.

Auto



POST Get data • POST Get data • My Collection + No Environment

My Collection > Get data Save Share

POST http://localhost:3000/api/v1/auth/login Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "email": "jpablo@farmacia.com",
3   "password": "miPassword123"
4 }
```

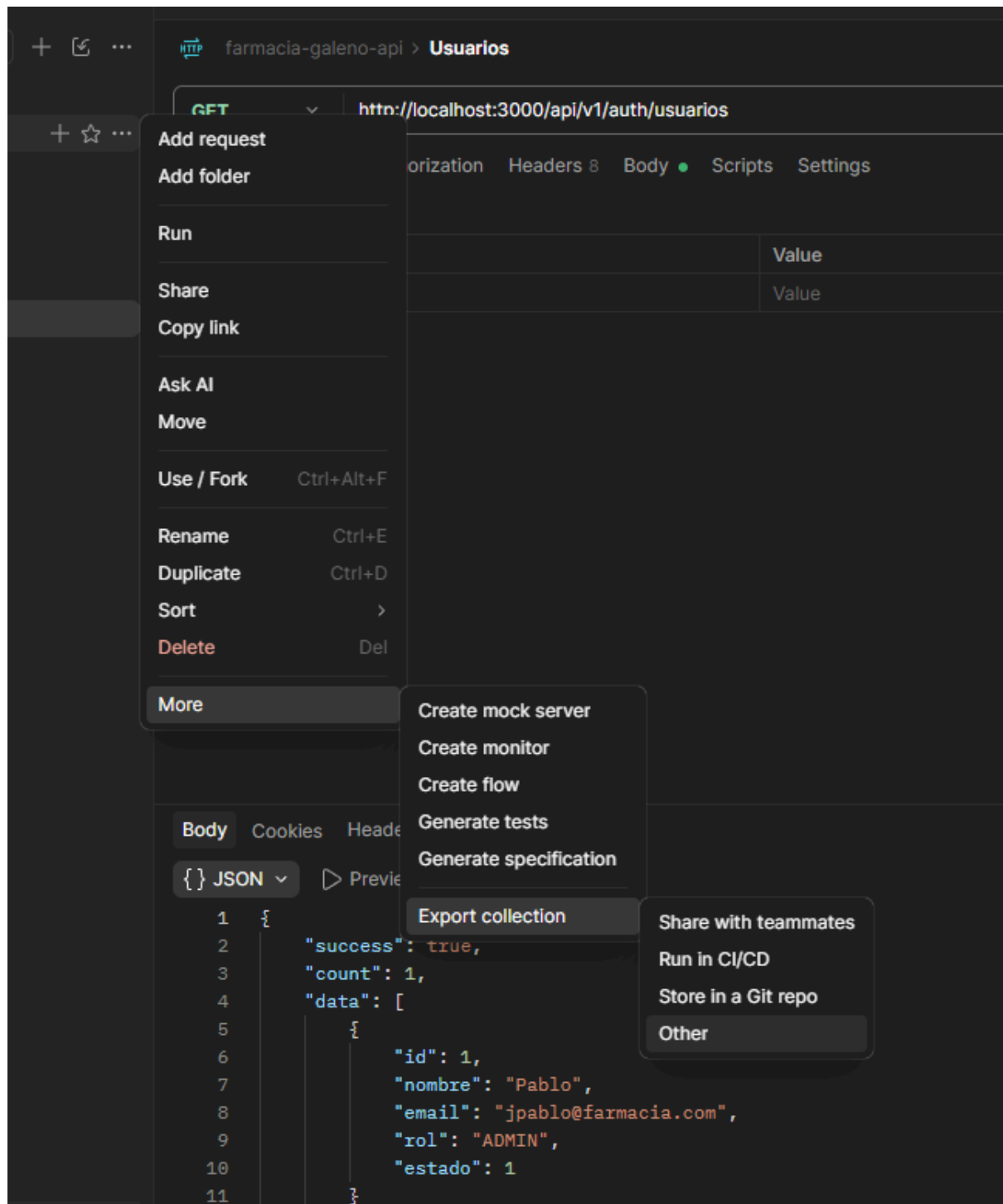
Body 401 Unauthorized • 5 ms • 364 B • Save Response

{ } JSON Preview Pa

1 {
2 "success": false,
3 "message": "Error
4 "error": "Usuario no encontrado"
5 }

401 Unauthorized

The request is unauthenticated.
[Pass the correct auth credentials.](#)



Vínculo a la Colección de Pruebas: [test-api/coleccion_postman.json](#)

```
{
  "info": {
    "_postman_id": "3e31119d-71f3-4fd1-a28c-f9c970d07115",
    "name": "farmacia-galeno-api",
    "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json",
    "_exporter_id": "58696799",
```

```

    "_collection_link":
    "https://go.postman.co/collection/58696799-3e31119d-71f3-4fd1-a28c-f9c970d07115?source
=collection_link"
  },
  "item": [
    {
      "name": "Productos",
      "request": {
        "method": "GET",
        "header": [],
        "url": {
          "raw": "http://localhost:3000/api/v1/productos",
          "protocol": "http",
          "host": [
            "localhost"
          ],
          "port": "3000",
          "path": [
            "api",
            "v1",
            "productos"
          ]
        }
      },
      "response": []
    },
    {
      "name": "Crear Producto",
      "request": {
        "method": "POST",
        "header": [],
        "body": {
          "mode": "raw",
          "raw": "{\r\n  \"nombre\": \"Paracetamol 500mg\",\r\n  \"descripcion\": \"Caja de 20
tabletas\",\r\n  \"precio\": 15.50,\r\n  \"stock\": 100,\r\n  \"categoria_id\": 1\r\n}",
          "options": {
            "raw": {
              "language": "json"
            }
          }
        }
      },
      "url": {
        "raw": "http://localhost:3000/api/v1/productos",
        "protocol": "http",
        "host": [
          "localhost"
        ],
        "port": "3000",

```

```

    "path": [
      "api",
      "v1",
      "productos"
    ]
  },
  "response": []
},
{
  "name": "Login correcto",
  "request": {
    "method": "POST",
    "header": [],
    "body": {
      "mode": "raw",
      "raw": "{\n  \"email\": \"jpablo@farmacia.com\", \n  \"password\": \"miPassword123\"\n}",
      "options": {
        "raw": {
          "language": "json"
        }
      }
    }
  },
  "url": {
    "raw": "http://localhost:3000/api/v1/auth/login",
    "protocol": "http",
    "host": [
      "localhost"
    ],
    "port": "3000",
    "path": [
      "api",
      "v1",
      "auth",
      "login"
    ]
  },
  "response": []
},
{
  "name": "Login incorrecto",
  "request": {
    "method": "POST",
    "header": [],
    "body": {
      "mode": "raw",
      "raw": "{\n  \"email\": \"jpablo@farmacia.com\", \n  \"password\": \"miPassword123\"\n}",

```



```
"options": {
  "raw": {
    "language": "json"
  }
},
"url": {
  "raw": "http://localhost:3000/api/v1/auth/login",
  "protocol": "http",
  "host": [
    "localhost"
  ],
  "port": "3000",
  "path": [
    "api",
    "v1",
    "auth",
    "login"
  ]
},
"response": []
},
{
  "name": "Usuarios",
  "request": {
    "method": "GET",
    "header": [],
    "body": {
      "mode": "raw",
      "raw": "{\n  \"email\": \"jpablo@farmacia.com\", \n  \"password\": \"miPassword123\"\n}",
      "options": {
        "raw": {
          "language": "json"
        }
      }
    }
  },
  "url": {
    "raw": "http://localhost:3000/api/v1/auth/usuarios",
    "protocol": "http",
    "host": [
      "localhost"
    ],
    "port": "3000",
    "path": [
      "api",
      "v1",
      "auth",

```

```

        "usuarios"
      ]
    }
  },
  "response": []
}
]
}

```

7. MATRIZ DE TRAZABILIDAD ÁGIL

Requisito / HU	Archivo BDD (.feature)	Pantalla Figma	Endpoint API	Componente Backend
HU-01	autenticacion.feature	Login Screen	POST /api/v1/auth/login	AuthController.js
HU-02	proveedores.feature	Proveedores Screen	GET /api/v1/proveedores	ProveedorController.js
HU-03	proveedores.feature	Registro de Proveedor	POST /api/v1/proveedores	ProveedorController.js
HU-04	compras.feature	Compras Screen	GET /api/v1/compras	CompraController.js
HU-05	compras.feature	Nueva Compra	POST /api/v1/compras	CompraController.js
HU-06	compras.feature	Detalle de Compra	GET /api/v1/compras/:id	CompraController.js
HU-07	inventario.feature	Inventario Screen	GET /api/v1/inventario	InventarioController.js
HU-08	inventario.feature	Actualización de Inventario	PUT /api/v1/inventario/:id	InventarioController.js

HU-09	inventario.feature	Detalle de Inventario	GET /api/v1/inventario/:id	InventarioController.js
HU-10	cuentas_por_pagar.feature	Cuentas por Pagar Screen	GET /api/v1/cuentas-por-pagar	CuentaPorPagarController.js
HU-11	cuentas_por_pagar.feature	Registro de Cuenta por Pagar	POST /api/v1/cuentas-por-pagar	CuentaPorPagarController.js
HU-12	cuentas_por_pagar.feature	Detalle de Cuenta por Pagar	GET /api/v1/cuentas-por-pagar/:id	CuentaPorPagarController.js
HU-13	consultas.feature	Consultas / Reportes Screen	GET /api/v1/consultas	ConsultaController.js
Requisito / HU	Archivo BDD (.feature)	Pantalla Figma	Endpoint API	Componente Backend

8. CONCLUSIONES Y LECCIONES APRENDIDAS

- La metodología ágil permitió organizar los requerimientos del sistema mediante épicas, historias de usuario, prioridades y Story Points, facilitando la definición del MVP.
- La utilización de BDD y Gherkin permitió convertir las historias de usuario en escenarios verificables, incluyendo casos exitosos y situaciones de error.
- La definición de reglas de negocio ayudó a establecer condiciones claras para procesos como proveedores, compras, inventario y cuentas por pagar.
- La trazabilidad entre historias de usuario, escenarios BDD, interfaz, API y backend permitirá mantener una relación clara entre los requerimientos y su implementación.
- El proyecto permitió comprender la importancia de planificar y organizar los requerimientos antes de iniciar el desarrollo técnico, reduciendo ambigüedades y facilitando las siguientes etapas de implementación.

9. REFERENCIAS

- Cucumber. (s. f.). *Gherkin reference*. <https://cucumber.io/docs/gherkin/reference/>
- Cucumber. (s. f.). *Introduction to Cucumber*. <https://cucumber.io/docs/>
- GitHub. (s. f.). *GitHub Docs*. <https://docs.github.com/en>
- OpenAPI Initiative. (2026). *OpenAPI Specification 3.2.1*.
<https://spec.openapis.org/oas/latest.html>
- World Wide Web Consortium. (2025). *Web Content Accessibility Guidelines (WCAG) 2.1*.
<https://www.w3.org/TR/WCAG21/>